

Reasoning Patterns for Efficient AI Coding: An Empirical Study

ANONYMOUS AUTHOR(S)

Large Reasoning Models (LRMs) increasingly produce long reasoning traces that are used not only to explain model decisions but also to guide prompting, self-refinement, and inference-time control. However, their underlying reasoning patterns in AI coding tasks remain poorly understood, limiting systematic analysis and optimization. We conduct an empirical study of reasoning patterns in AI coding by transforming reasoning traces into action sequences and mining reusable patterns at two levels: macro patterns that capture task-level reasoning strategies, and micro patterns that distinguish valid from invalid reasoning behaviors at a finer granularity. Our findings show that each coding task exhibits a distinct macro pattern reflecting its underlying problem-solving strategy; for example, code generation follows a design-to-implementation flow, whereas debugging follows an evidence-fix-recheck loop. At the micro level, valid traces exhibit verification-driven convergence, while invalid traces are characterized by anti-patterns such as repeated design revision, reflection, and knowledge recall without closure. We further demonstrate the utility of these patterns through three applications. Macro-pattern features enable accurate task identification from reasoning traces. Micro-pattern-guided prompting improves single-attempt correctness with comparable token cost, while anti-pattern-aware early stopping reduces reasoning tokens by 30–69% while maintaining competitive accuracy. These gains generalize across unseen models, indicating that the identified patterns are robust and model-agnostic.

Additional Key Words and Phrases: reasoning patterns, AI coding, reasoning trace analysis, reasoning pattern mining, inference-time optimization

ACM Reference Format:

Anonymous Author(s). 2026. Reasoning Patterns for Efficient AI Coding: An Empirical Study . 1, 1 (September 2026), 22 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Large reasoning models (LRMs) are rapidly reshaping AI coding [35, 36]. Unlike conventional code language models that directly generate outputs, LRMs explicitly produce multi-step reasoning before arriving at a solution. This capability is particularly valuable for coding tasks, which often require structured intermediate reasoning, such as algorithm design in code generation and state tracking in program execution reasoning. Consequently, reasoning has become a first-class component of coding performance rather than a by-product of final answers.

The availability of explicit reasoning traces has enabled extensive research on their role in code-related reasoning and generation [6, 29]. Prior work shows that reasoning traces can significantly influence coding outcomes, but they also introduce inefficiencies, including incorrect intermediate assumptions, redundant exploration, repetitive reflections, and excessive deliberation that increases inference cost without improving correctness [4, 12, 18, 39]. Reasoning traces have therefore been increasingly exploited beyond explanation, supporting self-correction [20, 22], evaluation [26, 30], and inference-time control [9, 10].

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM XXXX-XXXX/2026/9-ART

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

Despite these advances, existing work primarily focuses on final task performance [3, 15, 16], prompting strategies [13, 27, 29], or isolated reasoning behaviors in specific coding tasks (e.g., code generation) [12, 18, 31, 39], rather than treating reasoning traces as structured, recurring regularities across instances and tasks. As a result, the higher-level organizational structure of reasoning in AI coding remains underexplored. In particular, it is still unclear whether LRMs exhibit stable reasoning patterns across tasks, what distinguishes effective from ineffective reasoning trajectories, and how such patterns relate to reasoning efficiency and correctness.

To address this gap, we conduct an empirical study of reasoning patterns in AI coding. We collect reasoning traces generated by LRMs on four representative coding tasks (i.e., code generation, code execution reasoning, program debugging, and code translation) and transform them into structured reasoning action sequences using a unified action taxonomy. This abstraction enables us to move beyond unstructured textual traces and analyze reasoning as structured behavioral processes. Based on this representation, we study the structure, effectiveness, and practical utility of reasoning patterns across tasks and models.

Specifically, we investigate the following research questions:

- **RQ1 (Macro Patterns): What task-level reasoning patterns emerge across different coding tasks?**

Different coding tasks impose distinct reasoning demands, yet it remains unclear whether LRMs develop consistent task-level reasoning structures. To characterize these structures, we segment reasoning traces into action sequences using a predefined taxonomy and mine recurring action-fragment signatures across four coding tasks, yielding a high-level view of how reasoning strategies differ across tasks.

- **RQ2 (Micro Patterns): What fine-grained reasoning patterns distinguish valid and invalid reasoning trajectories?**

Beyond task-level structures, reasoning quality is largely shaped by fine-grained dynamics within trajectories. We therefore mine positive and anti-patterns from valid and invalid reasoning traces to capture local mechanisms that facilitate or hinder problem solving. For tasks with objective correctness signals, we assess trace validity using an ensemble LLM-as-a-judge protocol and extract micro patterns via trace measurement and frequent fragment mining.

- **RQ3 (Applications): How can discovered reasoning patterns be leveraged to improve AI coding performance?**

If reasoning patterns reflect stable and generalizable structure, they should be useful beyond analysis, enabling improvements in inference-time behavior. To this end, we study three applications: task identification from macro-pattern features, prompting augmentation with micro-pattern guidance, and inference-time efficiency improvement via anti-pattern-triggered early stopping.

Our study yields three main findings. First, each coding task exhibits a distinct macro-level reasoning structure: code generation follows a design-to-implementation flow, code execution reasoning relies on state tracing, debugging proceeds via an evidence–fix–verification loop, and code translation centers on semantic mapping. Second, valid and invalid trajectories diverge in their micro-level dynamics: valid traces show verification-driven convergence, whereas invalid traces exhibit repeated redesign, reflection, and knowledge recall without resolution. Third, these patterns are actionable: they enable accurate task identification, improve single-shot correctness via pattern-guided prompting, and reduce reasoning tokens by 30–69% via anti-pattern-aware early stopping while preserving competitive accuracy.

In summary, this paper makes the following contributions:

- We present the first empirical study of reasoning patterns in cross-task AI coding, introducing a reasoning-action representation that transforms free-form reasoning traces into structured action sequences for analysis.
- We identify both macro- and micro-level reasoning patterns across four representative coding tasks, revealing task-specific reasoning strategies as well as recurring fine-grained patterns and anti-patterns associated with valid and invalid reasoning trajectories.
- We show that reasoning patterns constitute an actionable abstraction for LRM-based AI coding, and demonstrate their utility in task identification, pattern-guided prompting, and anti-pattern-aware early stopping, improving both interpretability and inference efficiency.

2 Methodology

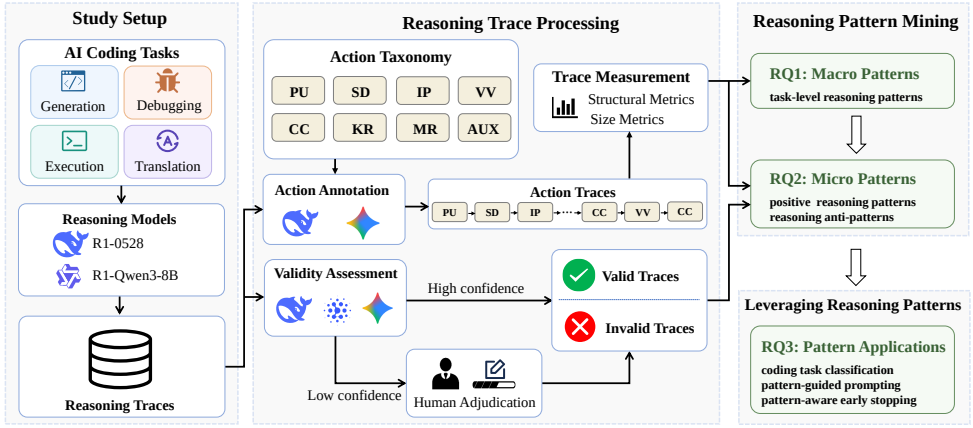


Fig. 1. Overview of the reasoning-pattern analysis framework.

Figure 1 illustrates the process of our empirical study. First, we collect reasoning traces from two LRMs on four representative AI coding tasks (§2.1) and assess the validity of each trace. Second, guided by the reasoning-action taxonomy, we annotate each reasoning trace into an action trace and quantify reasoning behaviors using action- and trace-level metrics (§2.2). Third, we mine recurring reasoning patterns from the resulting action traces (§2.3), covering task-level macro patterns (RQ1) and validity-related micro patterns and anti-patterns (RQ2). Finally, RQ3 explores how the discovered reasoning patterns can be leveraged for AI coding (§2.4), including coding task classification, pattern-guided prompting, and pattern-aware early stopping.

2.1 Study Setup

To investigate reasoning patterns in AI coding, we study four representative coding tasks that cover program synthesis, comprehension, maintenance, and transformation:

- *Code generation*, which requires a model to synthesize executable programs from natural-language specifications. We use LiveCodeBench v4–v6 [15], comprising 442 instances.
- *Code execution reasoning*, which requires a model to infer the output of a program through static reasoning rather than actual execution. We use the input–output prediction subset of CodeSense [21], comprising 382 instances.

- *Program debugging*, which requires a model to localize and repair defects in buggy programs. We use DebugBench [25], comprising 500 instances.
- *Code translation*, which requires a model to transform programs across programming languages while preserving their semantics. We use ClassEval-T [32], comprising 376 instances.

These tasks all provide objective correctness oracles, which allow us to analyze both reasoning traces and task outcomes under a unified experimental setting.

We evaluate two publicly available LRMs: DeepSeek-R1-0528 (denoted as R1-0528) and DeepSeek-R1-0528-Qwen3-8B (denoted as R1-Qwen3-8B) [6]. R1-Qwen3-8B is obtained by distilling R1-0528 into a Qwen3-8B backbone. For every task instance, both models are evaluated using identical prompts and decoding configurations. We collect each model’s complete reasoning trace together with its final response, forming the dataset used in the subsequent analyses.

2.2 Reasoning Trace Processing

This section describes how we process the collected reasoning traces into reasoning action traces and the derived metrics used for subsequent reasoning pattern mining and analysis.

2.2.1 Reasoning Trace Collection. For each task instance, we instantiate the benchmark-provided prompt template with instance-specific inputs (e.g., problem description, starter code, buggy program, or source-language snippet), and submit it independently to each model in thinking mode under a unified sampling configuration (temperature = 0.6, top- p = 0.95).

We extract the content enclosed by the <think> tags as the reasoning trace. To ensure completeness, we regenerate any response with an empty reasoning trace or lacking the information required for annotation or task evaluation, until a complete trace and task output are obtained.

After filtering incomplete, unparseable, or unlabeled responses, the resulting dataset contains 3,376 reasoning traces across the four tasks and two models: 442/440 traces for code generation, 377/378 for code execution reasoning, 499/496 for debugging, and 372/372 for code translation on R1-0528/R1-Qwen3-8B, respectively.

2.2.2 Reasoning Trace Validity Assessment. To compare valid and invalid reasoning behaviors in RQ2, we assess the validity of each reasoning trace using an ensemble LLM-as-a-judge protocol [17, 40]. Trace-level validity is distinct from task-level functional correctness: a response may produce a correct final answer while its reasoning trace still contains logical inconsistencies or unnecessary reasoning detours. A reasoning trace is considered *valid* if its reasoning is internally consistent and progresses coherently toward the final solution.

We employ three judge models, DeepSeek-V3.2 [7], Gemini-3-Flash [11], and GLM-5.1 [37], each independently classifying a trace as *valid* or *invalid*. To reduce judgment variance, each judge is queried three times, returning a binary label $y_{m,r} \in \{\text{valid}, \text{invalid}\}$ with a confidence score $c_{m,r} \in [0, 1]$.

We aggregate the nine judgments using confidence-weighted voting:

$$S(\ell) = \frac{\sum_{m \in \mathcal{M}} \sum_{r=1}^3 c_{m,r} \cdot \mathbf{1}[y_{m,r} = \ell]}{\sum_{m \in \mathcal{M}} \sum_{r=1}^3 c_{m,r}}, \quad (1)$$

where $S(\ell)$ denotes the normalized confidence-weighted support for label ℓ . The final label is assigned as $\hat{y} = \arg \max_{\ell} S(\ell)$, with confidence $\max_{\ell} S(\ell)$. We automatically accept the aggregated label when its confidence exceeds 0.8; otherwise, the reasoning trace is adjudicated by two human annotators. The resulting labels define the valid and invalid reasoning-trace groups. In total, the

Table 1. Taxonomy of Reasoning Actions.

Abbr.	Reasoning Action	Description
PU	Problem understanding	Understanding and clarifying the problem requirements.
SD	Solution design	Designing the high-level solution approach and strategy.
IP	Implementation planning	Planning the detailed implementation steps and logic.
CC	Code construction	Writing and refining the actual code.
VV	Verification and validation	Testing, debugging, and validating the solution.
KR	Knowledge retrieval	Retrieving and applying relevant knowledge.
MR	Meta-reasoning	Reflecting on and monitoring the reasoning process itself.
AUX	Auxiliary	Auxiliary statements that support the reasoning flow.

dataset contains 2,452 valid traces and 924 invalid traces: R1-0528 contributes 1,439 valid and 251 invalid traces, while R1-Qwen3-8B contributes 1,013 valid and 673 invalid traces.

2.2.3 Reasoning Action Taxonomy and Annotation. Reasoning traces are free-form natural-language text, making systematic comparison across tasks and models difficult. To enable structured analysis, we represent each reasoning trace as a sequence of fine-grained *reasoning actions*, where a reasoning action is defined as a contiguous text span expressing a distinct reasoning behavior.

Drawing on prior studies of LLM reasoning [12, 20, 22, 35], we define a taxonomy of eight reasoning actions that captures the major stages of code reasoning, ranging from task understanding and solution construction to verification and reflection. To strengthen its validity, the taxonomy is further inspected by two researchers with over five years of experience in code intelligence systems, who independently evaluated the coverage, granularity, and ambiguity of the action definitions and iteratively refined them to resolve overlaps. The resulting taxonomy provides a task-independent representation of reasoning behaviors across different coding tasks. Table 1 summarizes the eight reasoning actions.

Using this taxonomy, we automatically annotate each reasoning trace with DeepSeek-V3.2 [7] and Gemini-2.5 Pro [24]. Specifically, each reasoning trace is segmented into contiguous spans, and each span is assigned exactly one reasoning action. To ensure consistent annotations, we require the models to generate structured JSON outputs containing the assigned reasoning actions, corresponding text spans, and their ordering. The annotation prompt specifies (1) the definitions of the eight reasoning actions with representative examples, (2) segmentation guidelines covering boundary criteria, granularity, and special-case handling, (3) priority rules for resolving ambiguous cases, and (4) the required JSON output schema. The complete annotation prompt is included in our replication package.

To assess annotation reliability, two human annotators independently re-annotated a randomly sampled subset of reasoning traces. Their annotations achieved substantial agreement with the LLM-generated annotations (Cohen’s $\kappa = 0.816$) [5], indicating that the proposed taxonomy and annotation protocol produce reliable reasoning-action annotations.

2.2.4 Action Trace Construction. The annotated reasoning actions are converted into an *action trace*, defined as an ordered sequence of reasoning actions:

$$\tau = (s_1, s_2, \dots, s_n),$$

where each $s_i \in \mathcal{C} = \{PU, SD, IP, CC, VV, KR, MR, AUX\}$ denotes one reasoning action from the taxonomy, forming the canonical representation for all subsequent analyses.

To emphasize transitions between reasoning stages while reducing the influence of repeated elaboration within the same action, we further derive a *collapsed action trace* $\bar{\tau}$ by merging adjacent identical actions. For example,

$$(SD, SD, KR, SD, VV, VV) \rightarrow (SD, KR, SD, VV).$$

2.2.5 Action Trace Measurement. We characterize each reasoning action trace using two groups of metrics: *size metrics*, which quantify reasoning length and inference cost, and *structural metrics*, which capture the composition and transition dynamics of reasoning actions.

Size metrics quantify both reasoning length and inference cost. We report the raw action-trace length n and the collapsed action-trace length m , where their ratio n/m measures the extent of consecutive action repetition. We additionally report the reasoning token count as a model-agnostic measure of inference cost.

Structural metrics characterize both the composition of reasoning actions and the transitions between them. We first measure the allocation of each reasoning action c , defined as its proportion in an action trace τ :

$$d_c(\tau) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}[s_i = c]. \quad (2)$$

To characterize reasoning flow, we compute transition probabilities between adjacent actions on the collapsed action trace $\bar{\tau} = (\bar{s}_1, \dots, \bar{s}_m)$:

$$P_{ab} = \frac{\sum_{\tau} \sum_{i=1}^{m-1} \mathbf{1}[\bar{s}_i = a, \bar{s}_{i+1} = b]}{\sum_{\tau} \sum_{i=1}^{m-1} \mathbf{1}[\bar{s}_i = a]}. \quad (3)$$

We further summarize the diversity of action transitions using the entropy of the transition matrix:

$$H(P) = \sum_a \pi_a \left(- \sum_b P_{ab} \log_2 P_{ab} \right), \quad (4)$$

where π is the stationary distribution satisfying $\pi = \pi P$. Larger entropy indicates more diverse next-action choices.

2.3 Reasoning Pattern Mining

This section describes how we mine reasoning patterns from the extracted reasoning action traces and derived metrics. We characterize reasoning patterns at two granularities: *macro patterns* at the task level and *micro patterns* at the action level, corresponding to RQ1 and RQ2, respectively.

2.3.1 Reasoning Pattern Definition. A *reasoning pattern* is an action-fragment signature that frequently recurs across a collection of action traces. Formally, given an action-fragment signature g , let $\text{match}(g, \bar{\tau})$ denote that g occurs in the collapsed action trace $\bar{\tau}$. The support of g over a trace set T is defined as:

$$\text{support}(g, T) = \frac{|\{\tau \in T : \text{match}(g, \bar{\tau})\}|}{|T|}. \quad (5)$$

An action-fragment signature is considered a candidate reasoning pattern if its support exceeds a predefined minimum support threshold. We use these patterns as observable regularities in action traces rather than as direct evidence of the model's latent cognitive mechanism.

We represent reasoning patterns using different abstractions across granularities. *Macro patterns* capture high-level reasoning strategies through ordering relations between actions. We denote

$A \Rightarrow B$ when action A precedes action B in a trace, allowing arbitrary intermediate actions and thus capturing coarse-grained ordering dependencies.

Micro patterns capture fine-grained reasoning dynamics based on adjacent action transitions. We denote $A \rightarrow B$ as a direct transition between consecutive actions, and $A \leftrightarrow B$ as a short bidirectional loop where two actions alternate. These representations capture local reasoning structures and enable contrastive analysis of effective behaviors and failure-prone anti-patterns.

2.3.2 Macro Pattern Mining. Macro patterns capture task-level regularities in reasoning behavior. For each task, we extract recurring action-fragment signatures from collapsed action traces. A signature is defined as a frequently occurring action sequence, such as $PU \Rightarrow SD \Rightarrow IP \Rightarrow CC$, indicating that these actions tend to co-occur in this order within traces of the same task.

We consider three classes of signatures: contiguous action n -grams (length 4–5), non-contiguous ordered action paths, and short cyclic loops. A signature is considered task-discriminative if it is frequent within the task and relatively rare in other tasks. The derived metrics interpret these signatures: action allocation indicates which actions dominate a task, transition probabilities describe how it moves between actions, and transition entropy summarizes whether the reasoning process is compact or dispersed.

2.3.3 Micro Pattern Mining. Micro patterns capture fine-grained reasoning dynamics within traces. We analyze valid and invalid reasoning action traces from two perspectives to identify micro patterns and anti-patterns that characterize effective and ineffective reasoning behaviors.

The first perspective concerns *action allocation*. We define a *closure share* as $VV + CC + AUX$, where VV denotes verification, CC denotes code modification, and AUX denotes answer finalization. This metric captures the proportion of reasoning effort devoted to converging toward a solution. In contrast, we define a *drift share* as $SD + MR + KR$, where SD denotes design reopening, MR denotes reflection, and KR denotes knowledge recall, capturing behaviors that revisit or expand the reasoning space. We summarize their relative balance using a *closure-minus-drift* score, which quantifies whether a trace is dominated by convergence-oriented or divergence-oriented reasoning.

The second perspective focuses on *local structural patterns*. We extract contiguous action n -grams from collapsed action traces and represent them as local transitions $A \rightarrow B$ and short loops $A \leftrightarrow B$. These patterns capture fine-grained procedural regularities that are not visible at the aggregate allocation level: allocation metrics characterize whether reasoning tends toward convergence (closure) or exploration (drift), whereas local patterns reveal the concrete action-level fragments through which such tendencies are realized.

We first apply an effect-size screen: a candidate pattern must exceed a minimum support threshold in its enriched group and show a minimum support contrast against the comparison group. Patterns that are more frequent in invalid traces than in valid ones are treated as anti-patterns. We then audit the retained contrasts using two-sided Fisher’s exact tests with Benjamini–Hochberg correction [2].

2.4 Leveraging Reasoning Patterns

This section describes how we leverage the discovered reasoning patterns to improve AI coding performance. We evaluate their effectiveness in three representative scenarios for RQ3: coding task classification, pattern-guided prompting, and pattern-aware early stopping.

2.4.1 Coding Task Classification. We train a supervised probe on reasoning-pattern features extracted from action traces to predict the coding task performed by an LRM. These features operationalize the macro patterns identified in RQ1 by encoding how a trace allocates actions, transitions between them, and distributes them across reasoning stages.

In total, we construct 57 features organized into seven groups: (1) eight action-allocation features; (2) two features capturing trace length and compression; (3) twelve key transition-probability features; (4) three repetition features capturing self-repetition and VV - or SD -centered iterative behaviors; (5) twenty-four position-aware action-allocation features that model where actions occur in the trace; (6) four action-effort features; and (7) four aggregate features measuring verification density, design ratio, reflection ratio, and code-output ratio. The probe is restricted to reasoning-pattern features and does not access task descriptions, final answers, or any external metadata.

2.4.2 Pattern-Guided Prompting. We incorporate the micro patterns identified in RQ2 into task prompts, integrating positive patterns from valid traces with negative constraints from anti-patterns. Positive patterns encourage transitions toward implementation, verification, and closure, while anti-pattern constraints discourage undesirable loops such as $SD \rightarrow KR \rightarrow SD$ and $VV \rightarrow SD \rightarrow MR$, steering the model toward structured, goal-directed reasoning.

2.4.3 Pattern-Aware Early Stopping. We leverage anti-patterns identified in RQ2 as online indicators of reasoning drift to enable early termination of low-quality reasoning trajectories. The stopping policy operates on streaming outputs by incrementally mapping text into reasoning actions and maintaining a collapsed action trace over time.

At each closed segment, the policy evaluates the current trace prefix using two criteria. The first detects persistent drift, triggered by repeated design reopening, reflection, and knowledge recall patterns (e.g., $SD \rightarrow KR \rightarrow SD$, $VV \rightarrow SD \rightarrow MR$) across consecutive segments, and is attenuated when verification-centered closure motifs (e.g., $VV \rightarrow IP \rightarrow VV$, $IP \rightarrow VV \rightarrow AUX$) appear. The second detects convergence, triggered once reasoning enters a stable verification-centered closure phase. When either criterion is satisfied, reasoning is terminated and the model is prompted to produce a final answer conditioned on the current prefix, with further reasoning disabled.

3 RQ1: Macro Reasoning Patterns

3.1 Setup

RQ1 characterizes task-level reasoning behaviors using the macro pattern mining procedure described in Section 2.3.2. We analyze 3,376 collapsed reasoning action traces generated by two LRMs across four coding tasks. A macro pattern is retained if its within-task support exceeds a predefined threshold of 0.05 and its support difference from the other tasks exceeds a margin of 0.03. These hyperparameters are calibrated by two experts based on empirical inspection of preliminary results.

3.2 Results

Across the four coding tasks, we identify a distinct macro reasoning pattern for each task. Table 2 reports one representative macro signature per task and its within- versus cross-task support. The large support gaps (e.g., 91.4% vs. 17.0% for code generation and 45.9% vs. 2.2% for code execution reasoning on R1-0528) indicate that these signatures capture task-specific strategies rather than generic reasoning behaviors.

3.2.1 Code Generation Follows a Design-to-Implementation Pattern. Both models organize code generation as progressive refinement from high-level solution design to implementation planning and verification. R1-0528 interleaves design, planning, and verification ($PU \rightarrow SD \rightarrow$

Table 2. Macro Reasoning Patterns across Four Coding Tasks

Task	Macro Pattern	Representative Signature	Support (within-task vs. cross-task)		
			R1-0528	R1-Qwen3-8B	GLM-5.1
Generation	Design-to-implementation	$PU \Rightarrow SD \Rightarrow IP \Rightarrow VV$	91.4% vs. 17.0%	88.0% vs. 22.0%	66.7% vs. 10.5%
Execution Reasoning	State tracing	$PU \Rightarrow \{VV, MR\}^*$	45.9% vs. 2.2%	53.5% vs. 3.4%	54.3% vs. 17.1%
Debugging	Evidence-fix-recheck	$\{SD, IP\} \Rightarrow CC \Rightarrow VV$	68.1% vs. 21.5%	70.8% vs. 20.4%	70.9% [†] vs. 32.4%
Translation	Semantic mapping-to-implementation	$PU \Rightarrow SD \Rightarrow IP \Rightarrow CC$	36.5% vs. 9.5%	30.1% vs. 16.2%	51.7% [†] vs. 21.0%

[†] For GLM-5.1, the most discriminative signatures are $PU \Rightarrow VV \Rightarrow CC \Rightarrow VV$ for debugging and $PU \Rightarrow SD \Rightarrow CC \Rightarrow VV$ for translation.

$VV \rightarrow SD \rightarrow IP \rightarrow VV$), with $IP \leftrightarrow VV$ as the dominant local interaction. R1-Qwen3-8B follows the same overall structure but adds meta-reasoning ($PU \rightarrow SD \rightarrow MR \rightarrow SD \rightarrow VV$).

3.2.2 Code Execution Reasoning Follows a State-Tracing Pattern. Both models perform code execution reasoning by progressively simulating program states and validating intermediate results. R1-0528 follows a compact state-tracing and output-verification trajectory ($PU \rightarrow VV \rightarrow MR$) with the lowest transition entropy among all tasks (1.690). R1-Qwen3-8B follows the same overall strategy through longer verification-reflection cycles ($PU \rightarrow VV \rightarrow MR \rightarrow VV$), resulting in a more iterative process.

3.2.3 Program Debugging Follows an Evidence-Fix-Recheck Pattern. Both models organize debugging as an iterative cycle of fault validation, code modification, and re-verification. R1-0528 enters verification early and repeatedly alternates between IP and VV ($PU \rightarrow VV \rightarrow IP \rightarrow VV \rightarrow CC \rightarrow VV$), supported by frequent $PU \rightarrow VV$ and $CC \rightarrow VV$ transitions. R1-Qwen3-8B follows the same pattern but more often reopens SD after verification ($PU \rightarrow SD \rightarrow VV \rightarrow SD \rightarrow IP \rightarrow CC \rightarrow VV \rightarrow CC$), suggesting a more exploratory debugging strategy.

3.2.4 Code Translation Follows a Semantic Mapping Pattern. Both models begin by constructing semantic correspondences between the source and target programs before proceeding to implementation. R1-0528 transitions more directly from semantic mapping to implementation and validation ($PU \rightarrow SD \rightarrow IP \rightarrow VV \rightarrow IP$), whereas R1-Qwen3-8B revisits SD more frequently through $PU \rightarrow SD \rightarrow IP \rightarrow SD \rightarrow IP$. The shared signature $PU \Rightarrow SD \Rightarrow IP \Rightarrow CC$ indicates that code translation is dominated by semantic reconstruction followed by code construction, rather than simple textual transformation.

3.2.5 Shared Reasoning Scaffold. Despite their distinct objectives, all four coding tasks exhibit a common understand-then-act reasoning scaffold. PU serves as the entry point in more than 95% of reasoning traces across both models, after which reasoning diverges into task-specific sequences involving SD , IP , VV , and CC . VV remains central in the representative signatures of generation, execution reasoning, and debugging, while IP is especially prominent in generation and translation. The raw-to-collapsed length ratio remains relatively stable across tasks (1.3–1.7), indicating moderate local repetition while preserving similar high-level reasoning structures. Overall, task-specific behaviors emerge only after the shared problem-understanding stage.

3.2.6 Cross-Model Comparison. The two LRMs exhibit highly consistent macro reasoning patterns while differing in how these patterns are realized. Across all four tasks, R1-Qwen3-8B generates longer reasoning traces than R1-0528 (+37.1% for code generation, +133.1% for code execution reasoning, +21.8% for program debugging, and +7.6% for code translation) and consistently shows equal or higher transition entropy, particularly for execution reasoning and debugging. These results indicate that both models adopt the same task-level reasoning strategies, but R1-Qwen3-8B

Table 3. Length, Cost, and Allocation Statistics for Valid and Invalid Reasoning Traces

Metric	R1-0528		R1-Qwen3-8B	
	Valid	Invalid	Valid	Invalid
Reasoning tokens	3,690	6,609	4,330	6,991
Raw action length	18.58	24.96	22.64	32.21
Collapsed action length	11.78	14.91	15.97	21.99
Closure share	0.414	0.241	0.330	0.219
Drift share	0.247	0.391	0.313	0.464
Closure–Drift	+0.167	−0.150	+0.017	−0.245

realizes them through more iterative and dispersed trajectories, whereas R1-0528 follows more compact paths.

Finding 1. Across both LRMs, each coding task exhibits a distinct macro reasoning pattern that reflects its underlying problem-solving strategy. Code generation follows a design-to-implementation pattern, code execution reasoning relies on state tracing, program debugging follows an evidence-fix-recheck cycle, and code translation is driven by semantic mapping followed by implementation.

4 RQ2: Micro Reasoning Patterns

4.1 Setup

RQ2 characterizes fine-grained reasoning behaviors by comparing valid and invalid reasoning action traces using the micro pattern analysis procedure described in Section 2.3.3. We analyze 2,452 valid traces and 924 invalid traces generated by two LRMs across four coding tasks. A micro pattern is retained if its enriched-group support exceeds 0.08 and its cross-group support difference exceeds 0.06. These thresholds are calibrated by two domain experts based on empirical inspection of preliminary results.

4.2 Results

We first examine differences between valid and invalid reasoning action traces, and then present the resulting micro patterns and anti-patterns that characterize effective and ineffective reasoning trajectories. These analyses reveal how fine-grained action dynamics give rise to observable differences in reasoning quality.

4.2.1 Invalid Traces Shift Effort Toward Drift-oriented Actions. Table 3 reports length, cost, and allocation statistics for valid and invalid traces, aggregated over the four oracle-backed tasks. Across both models, invalid traces are consistently longer than valid traces in tokens, raw action length, and collapsed action length, indicating that invalidity reflects not low-effort or prematurely terminated reasoning, but additional steps that fail to converge toward a correct solution.

However, increased effort alone does not explain the validity gap; the more pronounced difference lies in how effort is distributed. Invalid traces systematically shift allocation from closure-oriented to drift-oriented actions: the closure share decreases substantially (R1-0528: 0.414 → 0.241; R1-Qwen3-8B: 0.330 → 0.219), while the drift share increases (R1-0528: 0.247 → 0.391; R1-Qwen3-8B: 0.313 → 0.464), reflecting a shift from verification and finalization toward repeated problem reinterpretation, design reconsideration, and knowledge retrieval.

At the level of aggregate structure, this effect is captured by the closure-minus-drift score, which reverses sign between valid and invalid traces (R1-0528: $+0.167 \rightarrow -0.150$; R1-Qwen3-8B: $+0.017 \rightarrow -0.245$), indicating that invalid reasoning is dominated by exploratory and revisiting behaviors rather than merely lacking verification.

Table 4. Micro Reasoning Patterns across Four Coding Tasks

Task	Model	Micro Pattern	Representative Signature
Generation	R1-0528	Plan-verify-close	IP→VV→AUX
		Plan-verify iteration	IP→VV→IP→VV→IP
		Knowledge-verify-reflect detour	KR→VV→MR
		Design-knowledge detour	SD→KR→IP
	R1-Qwen3-8B	Verify-plan-reverify	VV→IP→VV
		Extended verify-plan	VV→IP→VV→IP
		Knowledge-design loop	KR↔SD
		Reflection-knowledge loop	MR↔KR
	GLM-5.1	Understand-design-plan	PU→SD→IP
		Reflection-design reopening	MR→SD→VV→MR
Execution Reasoning	R1-0528	State-trace verification	PU→VV→MR→VV
		Understand-reopening reflection	MR→PU→MR
	R1-Qwen3-8B	State-trace verification	PU→VV→MR→VV
		Reflection-design reopening	MR→SD→{PU,VV}
	GLM-5.1	State-trace rechecking	PU→VV→MR→VV
		Reflection-knowledge detour	MR→KR→VV
Debugging	R1-0528	Change-verify-close	CC→VV→AUX
		Verify-design-reflect reopening	VV→SD→MR
		Design-reflect detour	SD→MR→IP
	R1-Qwen3-8B	Verify-plan-reverify	VV→IP→VV
		Knowledge-design reopening	SD→KR→SD
	GLM-5.1	Reflect-verify-repair	MR→VV→CC
		Reflect-change premature close	MR→CC→VV→AUX
Translation	R1-0528	Design-plan-construct	SD→IP→CC
		Design-reflect reopening	SD→MR→SD→MR→SD
		Verify-plan-construct detour	VV→IP→CC
	R1-Qwen3-8B	Understand-design-plan-reflect	PU→SD→IP→MR
		Design-reflect replanning	SD→MR→SD→IP
		Understand-design reopening	PU→SD→PU→SD
	GLM-5.1	Verify-construct-reverify	VV→CC→VV
		Plan-reflect-replan	IP→MR→IP
		Construct-reflect loop	CC↔MR

* The gray shaded cells are anti-patterns.

4.2.2 Micro Patterns Distinguish Productive Iteration from Reopening. We next examine whether these shifts are reflected in fine-grained local structures. Table 4 summarizes the discovered micro patterns and anti-patterns, while Figure 2 visualizes support contrasts of representative high-support patterns across task-model settings. The results show that these differences are also manifested at the micro-pattern level. For instance, in R1-Qwen3-8B on code generation, the positive pattern $VV \rightarrow IP \rightarrow VV$ appears in 46.6% of valid traces but only 19.6% of invalid traces. In contrast, the anti-pattern $KR \leftrightarrow SD$ appears in 56.4% of invalid traces but only 15.9% of valid traces. Similarly, for R1-0528 on debugging, the pattern $CC \rightarrow VV \rightarrow AUX$ is substantially more frequent in valid traces (21.7% vs. 4.0%), whereas $VV \rightarrow SD \rightarrow MR$ is strongly associated with invalid traces (20.0% vs. 2.7%).

These patterns reveal a clear distinction in iterative behavior. Productive iteration typically follows a verify-adjust-reverify or verify-commit structure, consistent with prior findings that execution and self-verification improve reliability in code reasoning [13]. In contrast, invalid traces exhibit repetitive transitions centered on design revision (SD), reflection (MR), or knowledge recall (KR), without progressing toward implementation or closure.

We further check representative support contrasts using two-sided Fisher’s exact tests with Benjamini-Hochberg correction, and report the complete support, count, p -value, and q -value evidence in the replication package. These results suggest that iteration per se is not predictive of correctness; rather, its target and direction determine whether it is productive or degenerative.

4.2.3 Cross-model Consistency with Task-dependent Strength. Finally, we examine the generality of the observed validity contrast across models and tasks. The closure-versus-reopening distinction is consistent across both R1-0528 and R1-Qwen3-8B, indicating that the identified reasoning patterns are not model-specific artifacts.

However, the strength of this contrast varies across settings. It is more pronounced in R1-Qwen3-8B, where invalid traces exhibit stronger cycles of design revision, reflection, and knowledge recall, suggesting that failure arises not from a lack of reasoning steps, but from persistent non-convergent exploration.

Across tasks, the effect is strongest in code generation, where invalid traces are substantially longer (17,163 vs. 7,102 tokens for R1-0528; 14,925 vs. 8,598 for R1-Qwen3-8B) and heavily concentrated in drift-oriented behaviors. Code execution and debugging tasks exhibit similar but more structured patterns, while translation shows weaker and less symmetric differences, making it a less definitive case for behavioral separation.

Finding 2. Valid and invalid reasoning traces differ less in length than in their micro patterns and anti-patterns, with valid traces showing verification-driven convergence and invalid traces exhibiting anti-patterns of repeated design, reflection, and knowledge recall without closure.

5 RQ3: Applications of Reasoning Patterns

To assess the practical utility of the discovered reasoning patterns for efficient AI coding, we conduct a set of targeted studies across code-related tasks.

- **RQ3.1** Are reasoning patterns discriminative enough to distinguish different AI coding tasks? (Section 5.1)
- **RQ3.2** Can pattern-guided prompting improve task performance under comparable token budgets? (Section 5.2)
- **RQ3.3** Can pattern-aware early stopping reduce reasoning cost while preserving correctness? (Section 5.3)

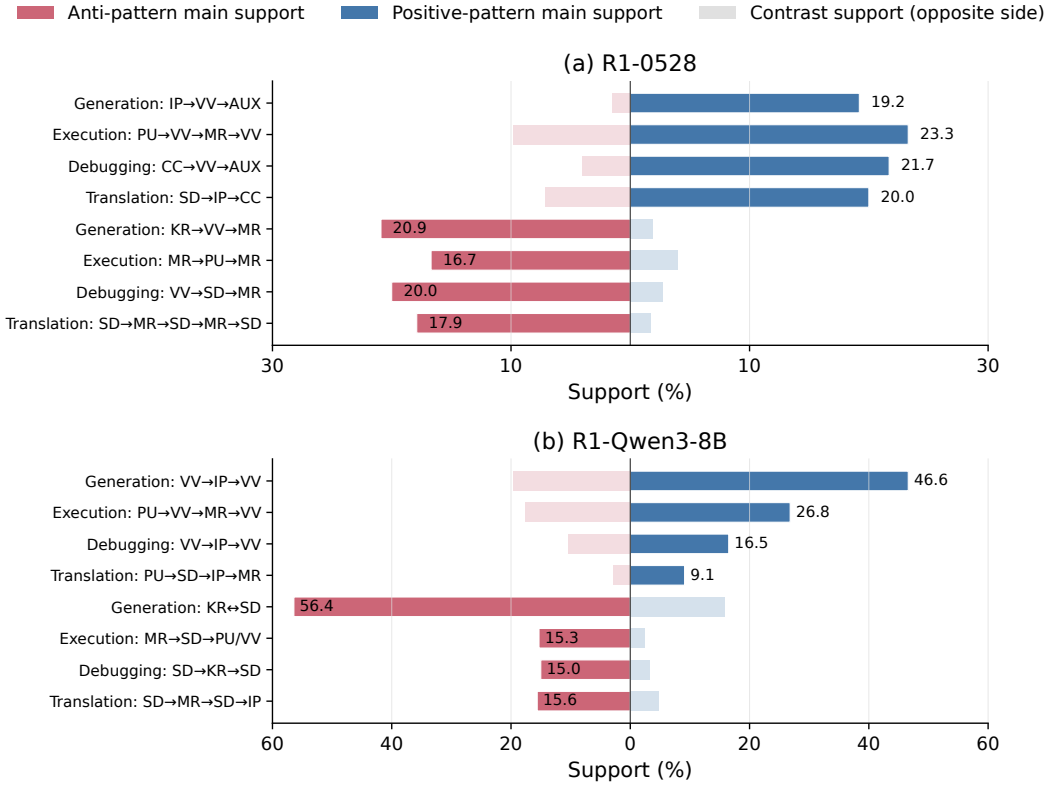


Fig. 2. Support comparison of micro reasoning patterns and anti-patterns.

5.1 Task Classification from Macro-pattern Features

Setup. Following the task recovery probe defined in Section 2.4, RQ3.1 evaluates whether task identity can be inferred from reasoning-pattern features alone. The test set consists of 3,376 reasoning traces generated by two models across four coding tasks. We evaluate two standard classifiers, logistic regression and random forest, using 5-fold stratified cross-validation. Performance is measured in terms of classification accuracy, reported as mean and standard deviation across folds; the random baseline is 25% for the four-class setting.

Results. Figure 3 reports task classification performance using only reasoning-pattern features derived from action traces. Error bars indicate cross-validation standard deviation, and the dashed line denotes the random baseline (25%).

Using 57 reasoning-pattern features, logistic regression achieves 0.861 ± 0.032 accuracy for R1-0528 and 0.770 ± 0.019 for R1-Qwen3-8B. Random forest yields comparable results (0.864 ± 0.027 and 0.775 ± 0.022 , respectively), indicating that task separability is robust across model families and classifiers. Since the models operate solely on reasoning-pattern features without access to task descriptions or outputs, these results demonstrate that reasoning traces preserve task identity in their structural pattern organization.

To understand which aspects of reasoning patterns drive this separability, we examine feature importance based on mutual information. For both models, the most informative features consistently relate to how reasoning effort is distributed across core actions (e.g., design ratio, and the

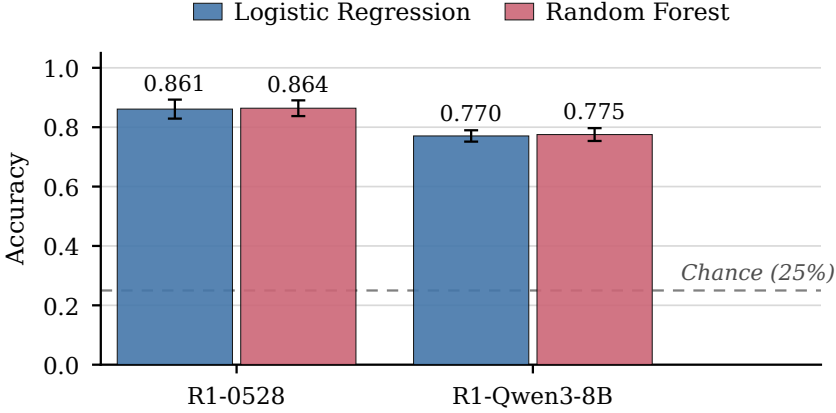


Fig. 3. Task identification from reasoning pattern features.

distributions of *PU*, *SD*, *IP*, and *VV*). Overall, task separability is primarily driven by how reasoning traces allocate and transition reasoning effort across key functional stages rather than by trace length alone, although length provides a secondary signal.

Finding 3.1. Reasoning-pattern features alone are sufficient to identify coding tasks with high accuracy, despite not using task content, answers, or metadata, suggesting that action traces inherently encode task-discriminative reasoning structures.

5.2 Micro-Pattern Guided Prompting

Setup. RQ3.2 explores whether micro reasoning patterns can be operationalized as inference-time guidance to improve coding performance. We compare two prompting strategies: a *default prompt*, which uses the original task instruction without additional guidance, and a *pattern-guided prompt*, which augments the same instruction with task-specific guidance derived from mined micro patterns.

We apply both prompting strategies to R1-0528 and R1-Qwen3-8B across four coding tasks with objective correctness oracles. To evaluate generalizability, we further replicate the same protocol on two additional models, DeepSeek-V4-Flash (DS-V4-F) [8] and GLM-5.1 [37].

Performance is measured using pass@1 [3]. To reduce variance from stochastic decoding, each setting is run three times, and we report mean pass@1 as the average across independent runs. We additionally report average per-instance token consumption (in thousands) to quantify inference cost.

Results. Table 5 summarizes the effectiveness of micro-pattern guided prompting across four AI coding tasks. We evaluate two settings: *Direct*, which applies patterns mined from the same model, and *Transferred*, which applies patterns mined from R1-0528 to unseen models (DS-V4-F and GLM-5.1).

Table 5. Performance and Cross-Model Transfer of Pattern-Guided Prompting

Task	Method	Direct				Transferred			
		R1-0528		R1-Qwen3-8B		DS-V4-F		GLM-5.1	
		P@1	Tok.	P@1	Tok.	P@1	Tok.	P@1	Tok.
Generation	Default	72.46	14.9	56.21	17.9	80.78	13.0	90.29	16.5
	Pattern-only	73.14	15.2	55.86	18.3	84.36	13.5	89.03	13.2
	Anti-pattern-only	72.08	14.6	56.39	17.6	79.84	12.5	89.15	13.0
	Pattern+Anti	74.49	15.6	56.88	17.3	87.80	12.2	89.16	12.9
Execution Reasoning	Default	55.76	1.7	44.24	3.4	56.81	2.1	59.84	1.6
	Pattern-only	57.16	2.4	48.17	4.4	58.21	2.6	60.32	1.6
	Anti-pattern-only	56.63	2.1	46.86	3.9	57.54	2.3	60.41	1.5
	Pattern+Anti	58.37	2.9	49.74	4.7	59.27	2.8	60.73	1.5
Debugging	Default	88.80	4.5	61.20	7.7	91.40	2.4	93.00	2.8
	Pattern-only	91.20	5.8	61.54	7.8	90.92	2.5	93.02	2.6
	Anti-pattern-only	90.47	5.1	61.08	7.6	92.18	2.2	93.11	2.6
	Pattern+Anti	94.60	6.6	61.72	7.9	92.80	2.3	93.20	2.5
Translation	Default	55.08	5.7	39.09	4.2	55.05	2.1	56.97	2.5
	Pattern-only	56.81	6.5	42.51	4.4	54.73	2.2	58.03	2.4
	Anti-pattern-only	57.44	6.2	43.27	4.3	56.73	1.8	58.52	2.4
	Pattern+Anti	58.78	7.3	44.14	4.5	57.44	1.9	58.75	2.3

1) *Direct Results.* Pattern-guided prompting improves mean pass@1 on all four tasks for both R1-0528 and R1-Qwen3-8B, with the largest gains on debugging for R1-0528 and on execution reasoning and translation for R1-Qwen3-8B(+5.80, +5.50, and +5.05 points, respectively).

Improvement is task-dependent, *tending to be larger on tasks with clearer separation between the positive patterns and anti-patterns identified in RQ2.* Token consumption remains comparable to the default prompt, *suggesting that the gains do not merely stem from longer reasoning.* These results *indicate* that structured positive and negative cues improve correctness without revealing task-specific solutions.

2) *Ablation: Positive Patterns vs. Anti-pattern Constraints.* To isolate the contribution of each guidance component, we further evaluate two ablated variants: *Pattern-only*, which retains only the positive patterns mined from valid traces, and *Anti-pattern-only*, which retains only the negative constraints derived from anti-patterns. Across the twelve task-model settings where ablations are available, each component alone yields inconsistent gains and can even underperform the default prompt: Pattern-only degrades R1-Qwen3-8B generation (55.86 vs. 56.21) and DS-V4-F translation (54.73 vs. 55.05), while Anti-pattern-only degrades DS-V4-F generation (79.84 vs. 80.78) and R1-Qwen3-8B debugging (61.08 vs. 61.20). In contrast, their combination achieves the highest pass@1 among all guided variants in every task-model setting and surpasses the default prompt

in all settings except GLM-5.1 generation. This complementarity is consistent with the RQ2 findings: positive patterns steer reasoning toward verification-driven closure (e.g., $VV \rightarrow IP \rightarrow VV$), whereas anti-pattern constraints suppress drift-oriented reopening loops (e.g., $SD \rightarrow KR \rightarrow SD$). Each ablated variant covers only one side of this contrast: Pattern-only steers reasoning toward closure but leaves the reopening loops unguarded, so reasoning can still drift into the failure modes that RQ2 associates with invalid traces. Conversely, Anti-pattern-only suppresses these loops but offers no constructive direction, so avoiding known failure loops does not by itself ensure productive closure. Combining the two addresses both sides and yields stable improvements.

3) *Transferred Results*. Pattern-guided prompting further generalizes to unseen models. On DS-V4-F, improvements are observed on all four tasks, with the largest gain on code generation (+7.02 points), with token usage remaining stable relative to the default prompt. On GLM-5.1, performance improves on execution reasoning, debugging, and translation, with only a minor decrease on code generation (−1.13 points), which we attribute to its near-saturated default accuracy (90.29%) that leaves limited headroom for external guidance; nonetheless, token consumption drops substantially (16.5K→12.9K), suggesting that the guidance accelerates convergence even when correctness is saturated. The transferred gains suggest that the mined micro patterns are model-agnostic rather than specific to the R1 family.

Finding 3.2. Micro-pattern-guided prompts that integrate positive patterns and anti-patterns consistently improve mean pass@1 across tasks under comparable token cost and generalize to held-out models, with the two guidance components being individually unstable but jointly complementary, suggesting that reasoning patterns provide transferable, solution-agnostic guidance for improving coding performance.

5.3 Pattern-aware Early Stopping

Setup. RQ3.3 examines whether anti-pattern signals can be leveraged for online early stopping to reduce inference cost while preserving correctness. We compare two settings: *Full Reasoning*, which allows the model to complete its full reasoning process before producing an answer, and *Pattern-aware Stop*, an online stopping policy defined in Section 2.4 that monitors the streaming reasoning trace and triggers early termination once sufficient drift signals are detected. Since Pattern-aware Stop requires access to streaming reasoning outputs, we apply it to R1-Qwen3-8B and the two held-out generalization models, while R1-0528 is excluded due to the lack of a streaming-capable interface during evaluation. Following RQ3.2, each setting is executed three times, and we report mean pass@1 together with the average token consumption per instance as a measure of inference cost.

Table 6. Pattern-aware Early Stopping Performance across Models

Task	Model	Full Reasoning		Pattern-aware Stop	
		P@1	Tok.	P@1	Tok.
Generation	R1-Qwen3-8B	56.21	17.89K	52.14 (−4.07)	10.64K (−40.56%)
	DS-V4-F	80.78	13.00K	73.36 (−7.42)	8.43K (−35.13%)
	GLM-5.1	90.29	16.54K	88.06 (−2.23)	7.85K (−52.52%)
Execution Reasoning	R1-Qwen3-8B	44.24	3.43K	40.58 (−3.66)	2.10K (−38.75%)
	DS-V4-F	56.81	2.13K	51.57 (−5.24)	0.85K (−59.78%)
	GLM-5.1	59.84	1.64K	58.63 (−1.21)	0.52K (−68.53%)
Debugging	R1-Qwen3-8B	61.20	7.65K	54.40 (−6.80)	4.18K (−45.32%)
	DS-V4-F	91.40	2.42K	86.60 (−4.80)	1.69K (−30.16%)
	GLM-5.1	93.00	2.77K	91.20 (−1.80)	1.74K (−37.16%)
Translation	R1-Qwen3-8B	39.09	4.18K	37.64 (−1.45)	2.33K (−44.22%)
	DS-V4-F	55.05	2.14K	50.79 (−4.26)	1.23K (−42.65%)
	GLM-5.1	56.97	2.53K	55.48 (−1.49)	1.51K (−40.56%)

Results. Table 6 summarizes the effectiveness of pattern-aware early stopping across four AI coding tasks. On R1-Qwen3-8B, the stopping policy consistently reduces token consumption by 38.75%–45.32%, with pass@1 reductions of 1.45 to 6.80 points. The cost–accuracy trade-off is task-dependent: translation exhibits the most favorable balance (44.22% token reduction with only a 1.45-point drop), whereas generation and debugging incur larger accuracy degradation at comparable savings, suggesting these tasks more often require extended reasoning before convergence.

We observe a similar trend on held-out models DS-V4-F and GLM-5.1, which were not involved in pattern mining. On DS-V4-F, the policy reduces token cost by 30.16%–59.78%, with the largest accuracy loss on generation (−7.42 points) but a more balanced trade-off on debugging. On GLM-5.1, token savings reach 37.16%–68.53% with small accuracy drops (1.21–2.23 points). These results demonstrate that pattern-aware early stopping generalizes beyond the models used for pattern discovery, while consistently exposing a controllable cost–correctness trade-off.

Finding 3.3. Pattern-aware early stopping achieves substantial token cost reduction (30–69%) with limited accuracy degradation (1.21–7.42 percentage-point pass@1 drop) across task–model settings, and generalizes across models.

6 Discussion

6.1 Understanding Reasoning Structures

Our results suggest that reasoning in AI coding is organized around recurrent action-level structures rather than isolated step sequences or linear chains of thought. Across tasks, we observe

stable macro-level behavioral regularities, indicating that reasoning traces exhibit compositional structure that is consistent within tasks but distinct across them.

At a finer granularity, the contrast between valid and invalid traces suggests that reasoning quality is closely related to how actions evolve toward convergence. This is reflected in the identified micro-level patterns and anti-patterns, which capture differences in action-level progression between successful and unsuccessful trajectories.

6.2 Future Research Directions

Our findings suggest several promising directions for future research on reasoning in AI coding.

6.2.1 Evaluation of Reasoning Models. Reasoning traces provide structured behavioral signals that are not captured by outcome-based metrics such as pass@1 or trace length. This motivates future work on evaluation frameworks grounded in action-level reasoning structure, enabling finer-grained assessment of model reasoning behavior.

6.2.2 Structure-Aware Inference Control. The discovered reasoning patterns suggest the potential for controlling inference through structural signals, enabling fine-grained monitoring and intervention during reasoning, including but not limited to early stopping.

6.2.3 Learning with Reasoning Structure. Beyond pattern-guided prompting, pattern-augmented reasoning traces expose intermediate structures that may serve as supervision signals for training models that explicitly optimize reasoning processes.

7 Threats to Validity

Construct Validity. Our analysis abstracts free-form reasoning traces into discrete reasoning actions, enabling systematic comparison but potentially losing fine-grained semantic distinctions and internal reasoning nuances. We mitigate this limitation by grounding the taxonomy in prior work, refining it via expert inspection, and validating automatic annotations against human re-labeling (Cohen’s $\kappa = 0.816$). However, alternative taxonomies, segmentation granularities, or collapsing rules may lead to different patterns, suggesting the need for sensitivity analysis in future work.

Internal Validity. Validity labels are derived from an LLM-based judge ensemble and task-level pass@1, which may still miss subtle reasoning errors or overestimate correctness. We reduce this risk using confidence-weighted voting, human adjudication of low-confidence cases, and objective correctness oracles. Nonetheless, prompting and early-stopping results should be interpreted as intervention evidence rather than causal attribution to specific mined patterns, motivating future controlled interventions for causal validation.

External Validity. Our study spans four coding tasks and four LRMs, which may not fully represent the diversity of software engineering scenarios or model architectures. While we evaluate both in-distribution and held-out models, further validation across tasks, languages, and model families is required to assess generality.

8 Related Work

Recent LRMs incorporate extended reasoning traces as a core component of inference [6]. In code-related settings, prior work shows that reasoning-before-coding is not uniformly beneficial and depends on how reasoning is structured relative to code generation [18]. Other studies explore leveraging external or structured signals to improve reasoning, such as mining discussion data into training chains [34] or validating intermediate steps via search-based refinement [28]. These efforts demonstrate the utility of explicit reasoning for code generation, but primarily focus on

improving outputs or training signals rather than characterizing recurring reasoning structures across tasks.

Prior work also examines reasoning quality and failure modes. Studies show that reasoning quality is not reliably aligned with final correctness, as flawed reasoning may still produce correct outputs and vice versa [39], and that superficial signals such as step count or verbosity are weak indicators of correctness [31]. Recent work further categorizes reasoning behaviors and failure patterns such as hallucination, overthinking, and erroneous backtracking [1, 12, 23]. While these studies move beyond outcome-only evaluation, they are often limited to single tasks or rely on coarse-grained or manual annotations, leaving open how reasoning structures vary across tasks and how they differ between valid and invalid traces.

Reasoning traces have also been studied for inference-time control. Methods such as SEER adapt reasoning strategies during generation [10], while work on overthinking motivates compressing redundant reasoning steps [4, 14, 38]. Early-stopping approaches exploit intermediate signals to terminate generation based on convergence or confidence criteria [19, 33]. These methods primarily optimize efficiency or decoding behavior, but do not explicitly derive reasoning patterns from empirical trace analysis.

In contrast, we treat reasoning traces as empirical behavioral artifacts, abstracting them into reasoning action traces and analyzing recurring structures across code generation, code execution reasoning, program debugging, and code translation. This enables a unified characterization of task-level patterns and action-level (anti-)patterns, and further connects these structures to downstream applications in task identification, pattern-guided prompting, and pattern-aware early stopping.

9 Conclusion

In this paper, we present an empirical study of reasoning patterns in AI coding. By abstracting reasoning traces into action sequences, we identify macro-level patterns that characterize task-specific reasoning strategies and micro-level patterns that distinguish effective from ineffective reasoning across four coding tasks. We further show that these patterns are actionable, enabling task identification, improved prompting, and token-efficient inference via early stopping, positioning reasoning patterns as a practical abstraction for understanding and improving the efficiency of LRMs in AI coding.

Data Availability

All code and data used in this study are publicly available at: https://anonymous.4open.science/r/ICSE_2027-2554.

References

- [1] Mohammad Abdollahi, Khandaker Rifah Tasnia, Soumit Kanti Saha, Jinqui Yang, Song Wang, and Hadi Hemmati. 2025. Demystifying errors in LLM reasoning traces: An empirical study of code execution simulation. *ACM Transactions on Software Engineering and Methodology* (2025).
- [2] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal statistical society: series B (Methodological)* 57, 1 (1995), 289–300.
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [4] Xingyu Chen, Jiahao Xu, Tian Liang, Zhiwei He, Jianhui Pang, Dian Yu, Linfeng Song, Qiuzhi Liu, Mengfei Zhou, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. [n. d.]. Do NOT Think That Much for 2+3=? On the Overthinking of Long Reasoning Models. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025 (Proceedings of Machine Learning Research)*. PMLR / OpenReview.net.

- [5] Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and psychological measurement* 20, 1 (1960), 37–46.
- [6] DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *CoRR* abs/2501.12948 (2025).
- [7] DeepSeek-AI. 2025. DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models. *CoRR* abs/2512.02556 (2025).
- [8] DeepSeek-AI. 2026. *DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence*. Technical Report. DeepSeek-AI. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf
- [9] Pengfei Gao and Chao Peng. [n. d.]. More with Less: An Empirical Study of Turn-Control Strategies for Efficient Coding Agents. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12-18, 2026, Rio de Janeiro, Brazil*.
- [10] Shuzheng Gao, Chaozheng Wang, Cuiyun Gao, and Michael R. Lyu. 2025. SEER: Enhancing Chain-of-Thought Code Generation through Self-Exploring Deep Reasoning. *CoRR* abs/2510.17130 (2025).
- [11] Google DeepMind. 2025. Gemini 3 Flash Model Card. <https://deepmind.google/models/model-cards/gemini-3-flash/>. Accessed: 2026-05-28.
- [12] Kevin Halim, Sin G. Teo, Ruitao Feng, Zhenpeng Chen, Yang Gu, Chong Wang, and Yang Liu. 2025. A Study on Thinking Patterns of Large Reasoning Models in Code Generation. *CoRR* abs/2509.13758 (2025).
- [13] Dong Huang, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. Codcot: Tackling code syntax errors in CoT reasoning for code generation. *CoRR* abs/2308.08784 (2023).
- [14] Kerui Huang, Shuhan Liu, Xing Hu, Tongtong Xu, Lingfeng Bao, and Xin Xia. 2025. Reasoning Efficiently Through Adaptive Chain-of-Thought Compression: A Self-Optimizing Framework. *CoRR* abs/2509.14093 (2025).
- [15] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. [n. d.]. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*.
- [16] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. [n. d.]. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- [17] Qintong Li, Leyang Cui, Lingpeng Kong, and Wei Bi. 2025. Exploring the Reliability of Large Language Models as Customized Evaluators for Diverse NLP Tasks. In *Proceedings of the 31st International Conference on Computational Linguistics*. Association for Computational Linguistics, Abu Dhabi, UAE, 10325–10344.
- [18] Renbiao Liu, Anqi Li, Chaoding Yang, Hui Sun, and Ming Li. [n. d.]. Revisiting Chain-of-Thought in Code Generation: Do Language Models Need to Learn Reasoning before Coding?. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*. PMLR / OpenReview.net.
- [19] Xin Liu and Lu Wang. [n. d.]. Answer Convergence as a Signal for Early Stopping in Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025*. Association for Computational Linguistics, 17896–17907. <https://aclanthology.org/2025.emnlp-main.904>
- [20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. [n. d.]. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- [21] Monoshi Kumar Roy, Simin Chen, Benjamin Steenhoeck, Jinjun Peng, Gail E. Kaiser, Baishakhi Ray, and Wei Le. [n. d.]. CodeSense: a Real-World Benchmark and Dataset for Code Semantic Reasoning. In *The Fourteenth International Conference on Learning Representations, ICLR 2026*. OpenReview.net. <https://openreview.net/forum?id=ehXVDJm0PS>
- [22] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. [n. d.]. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- [23] Zhongxiang Sun, Qipeng Wang, Haoyu Wang, Xiao Zhang, and Jun Xu. 2025. Detection and Mitigation of Hallucination in Large Reasoning Models: A Mechanistic Perspective. *CoRR* abs/2505.12886 (2025).
- [24] Gemini Team. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *CoRR* abs/2507.06261 (2025).
- [25] Runchu Tian, Yining Ye, Yujia Qin, Xin Cong, Yankai Lin, Yinxu Pan, Yesai Wu, Haotian Hui, Weichuan Liu, Zhiyuan Liu, and Maosong Sun. [n. d.]. DebugBench: Evaluating Debugging Capability of Large Language Models. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*. 4173–4198.

- [26] Xincheng Wang, Ruida Hu, Pengfei Gao, Chao Peng, and Cuiyun Gao. [n. d.]. An Agent-based Evaluation Framework for Complex Code Generation. In *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, Seoul, Korea, Republic of, November 16-20, 2025*. IEEE, 2427–2439.
- [27] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. [n. d.]. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.
- [28] Yibo Wang, Zhihao Peng, Ying Wang, Zhao Wei, Hai Yu, and Zhiliang Zhu. [n. d.]. MCTS-Refined CoT: High-Quality Fine-Tuning Data for LLM-Based Repository Issue Resolution. In *40th IEEE/ACM International Conference on Automated Software Engineering, ASE 2025, Seoul, Korea, Republic of, November 16-20, 2025*. IEEE, 1844–1855.
- [29] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. [n. d.]. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.
- [30] Qiujie Xie, Qingqiu Li, Zhuohao Yu, Yuejie Zhang, Yue Zhang, and Linyi Yang. [n. d.]. An Empirical Analysis of Uncertainty in Large Language Model Evaluations. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.
- [31] Haoran Xue, Gias Uddin, and Song Wang. [n. d.]. An Empirical Study of Reasoning Steps in Thinking Code LLMs. In *3rd ACM International Conference on AI-powered Software, AIware 2026*. <https://openreview.net/forum?id=2WgHyQZ2iO>
- [32] Pengyu Xue, Linhao Wu, Zhen Yang, Chengyi Wang, Xiang Li, Yuxiang Zhang, Jia Li, Ruikai Jin, Yifei Pei, Zhaoyan Shen, Xiran Lyu, and Jacky Wai Keung. 2025. ClassEval-T: Evaluating Large Language Models in Class-Level Code Translation. *Proc. ACM Softw. Eng.* 2, ISSTA (2025), 1421–1444.
- [33] Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Qiaowei Li, Minghui Chen, Zheng Lin, and Weiping Wang. [n. d.]. Dynamic Early Exit in Reasoning Models. In *The Fourteenth International Conference on Learning Representations, ICLR 2026*. <https://arxiv.org/abs/2504.15895>
- [34] Chengran Yang, Zhensu Sun, Hong Jin Kang, Jieke Shi, and David Lo. [n. d.]. Think Like Human Developers: Harnessing Community Knowledge for Structured Code Reasoning. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12-18, 2026, Rio de Janeiro, Brazil*.
- [35] Dayu Yang, Tianyang Liu, Daoan Zhang, Antoine Simoulin, Xiaoyi Liu, Yuwei Cao, Zhaopu Teng, Xin Qian, Grey Yang, Jiebo Luo, et al. 2025. Code to think, think to code: A survey on code-enhanced reasoning and reasoning-driven code intelligence in llms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 2586–2616.
- [36] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. [n. d.]. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*.
- [37] Z.ai. 2026. GLM-5.1 Model Card. <https://huggingface.co/zai-org/GLM-5.1>. Accessed: 2026-06-15.
- [38] Wenhao Zeng, Yaoning Wang, Chao Hu, Yuling Shi, Chengcheng Wan, Hongyu Zhang, and Xiaodong Gu. 2025. Pruning the Unsurprising: Efficient Code Reasoning via First-Token Surprisal. *CoRR* abs/2508.05988 (2025).
- [39] Binqian Zhang, Li Zhang, Zhiwen Luo, Yuxin Du, Fang Liu, Song Wang, and Lin Shi. 2026. Are They All Good? Evaluating the Quality of CoTs in LLM-based Code Generation. *IEEE Transactions on Software Engineering* (2026), 1928–1943. doi:10.1109/TSE.2026.3676295
- [40] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. [n. d.]. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.